

FPGA NxN CNN Convolution Accelerator

IEEE SSCS Egypt 2026 Student Design Competition — Specification Options,
Trade-offs & Recommendations

Target platform: ZedBoard (Zynq-7000, xc7z020)

Toolchain: Xilinx Vivado 2025.2

RTL language: VHDL | **Software / golden model:** Python

Design goal: 1 output pixel/cycle (mandatory) · every bonus item in-scope · FOM-maximizing

How to use this document: every section presents the available options with a full explanation, pros, cons, and a design-complexity rating, ending in a recommended choice. You make the final call on each — nothing here is committed until you say so.

Table of Contents

1.	2.	Foundational Concepts Primer
2.	3.	Input Pixel / Feature-Map Number Format
3.	4.	Kernel Coefficient Number Format
4.	5.	Accumulator Width & Overflow Budget
5.	6.	Rescale (Requantization) Policy — Shift Strategy
6.	7.	Rounding Convention (Truncation vs. Round-to-Nearest)
7.	8.	Overflow Handling (Saturation vs. Wraparound)
8.	9.	Boundary / Edge-Pixel Convolution Convention
9.	10.	Kernel-Size Parameterization Strategy
10.	11.	Multi-Channel / Multi-Kernel Parallelism Architecture
11.	12.	Core Convolution Compute Topology
12.	13.	Multiplier Implementation: DSP48 vs. LUT Fabric
13.	14.	Window Generation & Line-Buffer Memory Architecture
14.	15.	Pipelining & Retiming Strategy
15.	16.	ReLU Implementation & Control
16.	17.	Control-Plane Interface
17.	18.	Data-Plane Interface for the ZedBoard Demo
18.	19.	PS-Side Software Enablement for Python
19.	20.	Master Recommendation Table
20.	21.	Projected Table 1 Preview

2. Foundational Concepts Primer

A short, self-contained reference for the vocabulary used throughout this document. If any of this is already familiar, skip ahead — every later section can be read on its own, but leans on these definitions the first time each concept is used.

2.1 Binary Integers & Two's Complement

An unsigned N -bit binary number represents values 0 to $2^N - 1$ directly. To represent negative numbers, hardware almost universally uses **two's complement**: the most-significant bit (MSB) is treated as carrying weight -2^{N-1} instead of $+2^{N-1}$, giving a signed range of -2^{N-1} to $+2^{N-1} - 1$. To negate a two's-complement number, invert every bit and add 1. The reason hardware favors two's complement almost universally: the same adder circuit that adds two positive numbers also correctly adds signed numbers, including subtraction (implemented as adding a negated value) — no separate sign-and-magnitude logic is needed anywhere in the datapath.

2.2 Fixed-Point (Q-Format) Representation

A fixed-point number is stored as a plain integer, with a *conceptual* (not physically drawn) binary point placed at an agreed-upon position. The common shorthand is **Qm.f**, meaning m integer bits (the leftmost of which is the sign bit, if signed) and f fractional bits, for a total width of $m+f$ bits. The represented value is simply $\text{raw_integer} / 2^f$.

Example — Q1.7 signed, 8 bits total: 1 sign/integer bit, 7 fractional bits. Range: -1.0 to $+0.9921875$, resolution $1/128$. A raw byte of $0x40$ (64 decimal) represents $64/128 = 0.5$.

Example — Q8.0 unsigned, 8 bits total: 0 fractional bits — this is just a plain integer, e.g. a raw grayscale pixel value 0–255.

2.3 Why Fixed-Point Multiplication Doesn't Care About Your Q-Format

Multiplying two fixed-point numbers in hardware produces $\text{raw_result} = \text{raw_A} \times \text{raw_B}$ — a plain integer multiply, using exactly the same multiplier circuit (e.g. an 8×8 signed multiplier) no matter what Q-format you've assigned to A or B. The product's implied binary point automatically sits at $f_A + f_B$ fractional bits; interpreting the result correctly is a matter of *where you say the point is*, not anything the circuit itself does differently.

Key insight, referenced throughout this document: choosing a Q-format for your kernel weights costs *zero* extra hardware. The multiplier is identical whether you call your number Q1.7 or Q8.0. The only place a Q-format choice has any real hardware consequence is downstream, at the rescale/shift stage (Section 6) — which is exactly why several sections below (4, 6) treat number-format choice as essentially free, and defer the real decision to the shift amount.

2.4 FPGA Resource Primitives

Primitive	What it is
-----------	------------

LUT (Look-Up Table)	The basic fabric building block — a small memory table that implements combinational logic. The fundamental, most flexible, and (per the FOM formula) cheapest-weighted resource.
FF (Flip-Flop)	A single-bit register — the storage element used for pipelining. Not counted at all in the FOM formula.
DSP48E1	A dedicated hardened arithmetic block (present on the ZedBoard's xc7z020, 220 available) containing a 25×18 signed multiplier plus a 48-bit adder/accumulator and optional internal pipeline registers — far faster and denser than an equivalent fabric-built multiplier, but weighted 50× in the FOM's resource score.
BRAM (Block RAM)	Dedicated dual-port memory primitive, allocated in fixed 18Kb/36Kb blocks — efficient for large storage, but “quantized”: you pay for a whole block even if you use a fraction of it. Weighted 100× in the FOM's resource score — the harshest single term in the formula.
SRL (Shift Register LUT)	A special configuration mode on Xilinx 7-series LUTs that turns a single 6-input LUT into a 16- or 32-bit shift register — extremely area-efficient for exactly the kind of fixed-length delay line a convolution line buffer needs. Counted as ordinary LUTs in the FOM formula.
Distributed RAM / LUTRAM	LUTs reconfigured to act as small, addressable local memories — useful for register-file-sized, randomly-addressed storage. Also counted as ordinary LUTs.

2.5 Restating the FOM as a Design-Optimization Target

$$\text{FOM} = \text{Throughput} / [\text{Power} \times (\text{LUTs} + 50 \cdot \text{DSPs} + 100 \cdot \text{BRAMs})]$$

Since throughput is fixed at 1 output pixel/cycle (mandatory, non-negotiable per your instructions), the formula effectively reduces to $\text{FOM} \propto 1 / (\text{Power} \times \text{Resource_score})$, where $\text{Resource_score} = \text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs}$. Maximizing FOM therefore means minimizing power *and* minimizing this weighted resource score — and critically, DSPs and BRAMs are punished far more harshly per-unit than LUTs. This reframes several of the sections below (13 and 14 especially, both flagged **HIGH FOM LEVERAGE**) from ordinary implementation details into direct, deliberate FOM-optimization decisions.

3. Input Pixel / Feature-Map Number Format

The spec mandates unsigned fixed-point input data but leaves the exact width/format up to you, provided you justify it. This decision only affects the *interpretation* of the raw bits — per Section 2.3, it costs nothing extra in hardware by itself. What matters is picking the option that best matches the actual data you're processing and doesn't waste width downstream.

Option A — 8-bit unsigned integer (Q8.0)

Complexity: Low

✓ RECOMMENDED

Treat each input sample as a plain unsigned byte, 0–255 — the native representation of essentially every 8-bit grayscale image file format in existence.

PROS

- Matches the universal grayscale-image convention — zero conversion needed between a PNG/JPEG loaded in Python and your test-vector files.
- Minimum possible input bit-width → minimum resource/wiring cost for every line buffer, shift register, and multiplier input downstream.
- Trivial, bullet-proof report justification: “this is the native representation of 8-bit grayscale intensity data.”

CONS

- Only 256 distinct intensity levels — irrelevant for the mandatory 32×32 grayscale spec, but a mild limitation if you ever wanted to process higher-dynamic-range feature-map data.

Why recommended: Simplest, standard, matches the spec's own minimum, costs nothing, and is instantly recognizable to a judge inspecting your test files.

Option B — Normalized unsigned fixed-point (Q0.8)

Complexity: Low

Same 8 raw bits as Option A, but interpreted as an all-fractional value (raw 255 $\approx$ 0.996), mirroring how ML frameworks normalize images to [0,1] before quantization.

PROS

- Aligns with how a “real” trained-CNN quantization flow would normalize inputs, which reads well if you want to frame this as CNN-accelerator design practice.

CONS

- Per Section 2.3, this is *only* a bit-interpretation difference from Option A — the hardware is identical either way.
- Buys nothing tangible for a single, hand-specified convolution kernel (not a trained multi-layer network); adds interpretation overhead in the report/golden model for no benefit.

Option C — Wider input width (10 or 12-bit unsigned)

Complexity: Medium

Use more raw bits per sample for extra dynamic range, useful if the input is genuinely an intermediate feature map rather than an 8-bit image.

PROS

- More headroom/precision if this engine is later reused on non-image activation data.

CONS

- Costs real resources everywhere downstream: wider line-buffer shift registers, wider multiplier inputs, wider accumulator — directly works against the FOM-minimization goal for zero spec-driven necessity.
- Complicates the demo: no natural 10/12-bit source image format to draw test data from.

DECISION

8-bit unsigned integer (Q8.0) — the plain, native grayscale-pixel representation.

4. Kernel Coefficient Number Format

The spec fixes the kernel word at 8-bit signed fixed-point or integer — that part isn't a choice. What *is* a choice is where you conceptually place the binary point within that 8-bit signed word, and (per Section 2.3) that choice is completely free in hardware — the multiplier circuit is identical no matter which Q-format you declare.

Option A — Pure signed integer (Q7.0) Complexity: Low

Raw values $-128 \dots 127$ taken literally as whole numbers — matches classic hand-designed kernels (Sobel: $-1, 0, 1 / -2, 0, 2 / -1, 0, 1$) directly, with no scaling.

PROS

- Trivial match to classic edge-detection/sharpen/blur kernels needed for the mandatory edge-detection bonus demo.
- Simplest possible golden-model code and report explanation.

CONS

- Can't faithfully express a small fractional weight (e.g. a normalized 0.37 tap from a trained-style filter) without an implicit external scale factor anyway.

Option B — Signed Q1.7 fixed-point Complexity: Low

Interpret the 8-bit word as 1 sign/integer bit + 7 fractional bits, range -1.0 to $+0.9921875$ — the conventional target of typical int8 ML-quantization flows.

PROS

- Generalizes cleanly if you (or a judge) later load an arbitrary, trained-style fractional kernel.
- Signals awareness of standard ML-accelerator quantization convention in the report.

CONS

- Classic small-integer kernels (Sobel etc.) need pre-scaling before loading to fit the ± 1.0 range, adding a software-side conversion step.

Option C — Raw signed-integer storage, format deferred to the programmable rescale shift

Complexity: Low

✓ RECOMMENDED

Store coefficients as plain signed 8-bit integers (literally Option A's representation) at the hardware boundary, and let the per-kernel programmable rescale shift (Section 6) carry the actual effective Q-format — nothing about the kernel's true fixed-point meaning is baked into synthesis.

PROS

- Zero extra hardware over Option A.
- Supports Sobel-style integer kernels *and* normalized-style kernels on the same unmodified datapath — the shift value is what changes, not the RTL.
- The kernel-loading software (trivial Python) computes the correct shift per kernel; the hardware never needs to know or care.

CONS

- Slightly more responsibility on the coefficient-generation script to compute the right shift value — a non-issue in software.

Why recommended: This is the cleanest illustration in the whole document of Section 2.3's core insight: the Q-format is free, only the shift stage has any real cost. It gives you Option A's simplicity for the demo kernels and Option B's generality for arbitrary kernels, simultaneously, for nothing.

DECISION

Store kernel coefficients as raw signed 8-bit integers; their effective fixed-point interpretation is carried entirely by the per-channel programmable rescale shift (Section 6), not fixed at synthesis time.

5. Accumulator Width & Overflow Budget

This governs how wide the running sum inside the adder tree needs to be so that an intermediate partial sum can never silently overflow before you reach the final, deliberate rescale/saturate stage (Sections 6–8). Getting this wrong isn't a quality issue — it's a correctness bug, and correctness is the competition's judging criterion #1.

Option A — No margin: accumulator width = output width Complexity: Low

Use your final 16(+)-bit output width as the running-sum width throughout the entire adder tree.

PROS

- Minimum possible FF/LUT cost for every register in the adder tree.

CONS

- Dangerous: intermediate partial sums can genuinely exceed the final range even when the true final (rescaled) result would have fit — truncating mid-computation computes the *wrong number*, not just a less precise one.
- This is a correctness failure mode, not a quality trade-off.

Option B — Exact worst-case width by formula Complexity: Medium ✓ RECOMMENDED

Derive the minimum accumulator width that can never overflow, for any legal input, using $\text{acc_width} = \text{pixel_width} + \text{kernel_width} + \text{ceil}(\log_2(N^2)) + 1$. Worked example for $N=3$ (9 taps, 8-bit×8-bit products): $8 + 8 + \text{ceil}(\log_2(9)) + 1 = 8+8+4+1 = 21$ bits (round up to 24 for convenience). This guarantees zero possibility of intermediate overflow for any legal kernel and any legal image, however extreme.

PROS

- Mathematically guaranteed correctness — directly serves the competition's #1 judging criterion.
- Still essentially free: a handful of extra flip-flops on an adder tree you're building anyway. FFs aren't in the FOM's penalty formula at all.

CONS

- A few more registers/wires than the bare minimum — irrelevant given the near-zero FOM impact.

Why recommended: Guaranteed-correct at negligible cost. There is no real reason to skimp here — the resources this consumes (LUTs/FFs) are the cheapest currency in the whole FOM formula.

Option C — Generously over-provisioned width (always 32/48-bit)

Complexity: Low

Always carry the DSP48E1's native 48-bit accumulator width through the whole pipeline, regardless of N, for uniformity.

PROS

- Simplest possible VHDL generic (no per-N width derivation).
- “Free” if using DSP48-internal accumulation, since that's the block's native width anyway.

CONS

- Wastes register/wiring resources if this width propagates into fabric-based adder-tree/rescale logic beyond the DSP itself — modest, but avoidable.

DECISION

Use the exact worst-case formula (Option B) for the fabric adder-tree width. If any DSP48-internal accumulation is used for a partial-sum segment (see Section 13), it's fine to inherit that block's native 48-bit width for just that segment, since it's free.

6. Rescale (Requantization) Policy — Shift Strategy

The correctly-wide accumulator (Section 5) must be brought back down to the final signed output width. This section decides *how much* to shift — fixed at synthesis time, or programmable per kernel.

Option A — Fixed compile-time shift amount Complexity: Low

Pick one shift amount before synthesis and bake it in permanently for every kernel that will ever be loaded.

PROS

- Simplest possible rescale hardware — a literal fixed bit-select, not even a barrel shifter.
- Zero control-plane complexity.

CONS

- Forces every kernel you ever load to share one implicit gain — an integer Sobel kernel and a normalized-style kernel have very different natural output magnitudes, so one would need artificial coefficient pre-scaling to fake a matching gain.
- Directly works against the mandatory multi-kernel requirement (Section 11), where different simultaneous channels naturally want different gains.

Option B — Programmable per-kernel shift amount Complexity: Medium ✓ RECOMMENDED

Add one small control register (e.g. 5 bits) loaded alongside each kernel's coefficients, and implement the rescale as an arithmetic right-shift by that register's value via a small barrel/funnel shifter.

PROS

- Each of the mandatory K parallel kernel channels (Section 11) carries its own independently-calibrated gain — e.g. Sobel on channel 0, a smoothing filter on channel 1, both correctly scaled simultaneously.
- Costs only a small mux-based shifter and one tiny register — trivially cheap in LUTs, the cheapest resource in the FOM formula.

CONS

- Marginally more control logic than a fixed shift — a well-understood, standard structure.

Why recommended: The mandatory multi-kernel requirement essentially forces this choice on its own; the fixed-shift alternative would need software-side coefficient distortion tricks elsewhere for no real savings.

DECISION

Programmable per-channel shift amount, stored in a small control register alongside each channel's kernel coefficients.

7. Rounding Convention (Truncation vs. Round-to-Nearest)

When the rescale shift (Section 6) discards the low bits of the accumulator, do you simply drop them, or adjust the result toward the nearest representable value first?

Option A — Truncation (plain arithmetic right-shift) Complexity: Low

output = accumulator >>> shift, discarding the low bits outright. This is floor-rounding, not true truncation-toward-zero — it biases negative results further negative.

PROS

- Zero additional hardware — this is what a plain shift already does, for free.

CONS

- Introduces a small, systematic (statistically biased) error, always rounding toward negative infinity — visually shows up as a slight but consistent darkening across the whole output feature map.

Option B — Round-half-up Complexity: Low ✓ RECOMMENDED

Add a rounding constant equal to half the discarded weight before truncating: output = (accumulator + (1 << (shift-1))) >> shift. This is the standard convention used in the overwhelming majority of real fixed-point/DSP hardware.

PROS

- Essentially unbiased, noticeably better numerical/visual fidelity for the edge-detection demo image.
- Textbook-standard convention — easy to cite and easy to mirror exactly in the NumPy golden model for bit-exact verification.

CONS

- Costs one extra add before the shift — on an adder tree you already have; negligible.

Why recommended: Meaningfully better numerical fidelity than truncation, for the cost of a single add on logic you're already building.

Option C — Round-to-nearest-even (“banker's rounding”) Complexity: High

Like Option B, but exact-tie cases round to whichever result has an even last bit, matching IEEE-754's default rounding mode.

PROS

- Statistically the most unbiased convention of the three, in principle.

CONS

- Needs to detect the exact tie condition and conditionally branch on the truncated LSB — a genuinely subtler hardware corner case and a whole extra golden-model branch to verify bit-exactly.
- For a single conv layer (not a deep multi-layer network where bias could compound), the practical benefit over round-half-up is negligible relative to the added verification surface.

DECISION

Round-half-up.

8. Overflow Handling (Saturation vs. Wraparound)

After rescaling, can the result still exceed the final output word's representable range? The spec explicitly requires you to explain this — it's judged, not optional.

Option A — Wraparound Complexity: Low

Take the low *output_width* bits of the rescaled result and discard the rest, letting two's-complement overflow happen silently.

PROS

- Zero extra hardware.

CONS

- Produces genuinely, wildly wrong output values when triggered — a bright pixel can wrap to a dark or negative one. Any judge testing an adversarial/high-gain kernel (which the spec's own wording strongly implies they will) catches this instantly by comparing against their own reference.

Option B — Saturation (clamp to range) Complexity: Medium ✓ RECOMMENDED

Compare the rescaled result against the representable min/max (e.g. ± 32767 for 16-bit signed); if it's outside that range, output the boundary value instead. This is exactly what “clipping” means in every image/DSP tool.

PROS

- Matches universal image/DSP convention; output stays sensible even for adversarial kernels.
- Directly, explicitly answers the spec's own call-out on overflow handling with the professionally expected convention.
- Cheap: a magnitude/sign compare plus a 2:1 mux, using guard bits you already carried through the accumulator (Section 5).

CONS

- Negligible added comparator/mux logic.

Why recommended: The professionally expected, spec-anticipated answer, at negligible hardware cost.

DECISION

Saturating arithmetic on the final rescaled output.

9. Boundary / Edge-Pixel Convolution Convention

For an $N \times N$ kernel, the outer $(N-1)/2$ rows/columns near each image edge lack a complete surrounding window. This decision governs what happens there — and it's explicitly one of the “assume and document” items the instructions invite you to define yourself.

Option A — “Valid” convolution (no padding)

Complexity: Low

✓ RECOMMENDED

Simply don't produce output pixels where the window would run off the image edge. Output size shrinks to $(W-N+1) \times (H-N+1)$; every pixel you do produce is backed by fully real input data.

PROS

- Simplest possible window-generator hardware — no edge muxing, no fake-pixel injection anywhere.
- Zero ambiguity about what value to fabricate at the border, because nothing is fabricated.
- Cleanly satisfies the “state your assumptions” instruction without inventing data the spec never gave you.
- Frees up control-logic complexity budget for the topology/multiplier decisions where it actually moves the FOM needle.

CONS

- Output image is a few pixels smaller than the input in each dimension — needs to be clearly documented so dimensions aren't confusing.

Why recommended: Simplest, most defensible, least control logic, and directly matches the instructions' own invitation to state assumptions rather than guess at unstated conventions.

Option B — Zero-padding (“same” output size)

Complexity: Medium

Conceptually surround the input with a border of zero-valued pixels so the output feature map matches the input size exactly — the standard convention in most CNN-framework literature.

PROS

- Output size exactly matches input size, a framework-familiar convention.

CONS

- Real added control complexity: edge detection plus zero-injection at all four borders, with extra handling at the actual corners.
- Introduces a visible dark/artificial border in the demo output image — a zero-padded edge-detection result shows a literal black frame around the real detected edges, which looks worse in a live demo than an honestly smaller image.

Option C — Edge replication / reflection padding Complexity: High

Fill missing window taps by replicating the nearest real edge pixel, or mirror-reflecting the image.

PROS

- Best visual border quality of the three options — no artificial dark frame.

CONS

- By far the most complex window-generator logic here — several extra mux layers to track and select the correct replicated/reflected source per edge and corner case — for a benefit (marginally nicer border) with zero bearing on any judged criterion.

DECISION

“Valid” convolution (no padding); the resulting $(W-N+1) \times (H-N+1)$ output size is documented explicitly as a stated design assumption.

10. Kernel-Size Parameterization Strategy

The spec calls for “an $N \times N$ convolution kernel” with programmable *coefficients*. This decision is about whether N itself needs to be switchable at runtime inside one bitstream, or whether it's fixed per build and generalized architecturally.

Option A — Runtime-variable N in one fixed bitstream Complexity: High

Build the window generator, MAC array, and adder tree to the maximum supported size (e.g. $5 \times 5 = 25$ taps), and mask off the unused outer taps when running in a smaller mode (e.g. 3×3).

PROS

- A single bitstream can demonstrate size-flexibility live, which can look impressive in a demo.

CONS

- Permanently pays the full worst-case resource cost (25-tap adder tree, 25 multipliers per channel) even when running in the cheaper 3×3 mode — a direct, substantial, unnecessary FOM resource-term hit.
- Adds real control complexity and a whole extra bug class (does the tap-masking correctly zero every contribution, in every mode?), and the golden model must replicate the exact same masking bit-for-bit.
- Timing closure gets harder because the adder tree is always sized for the worst case, even in the cheaper mode.
- The spec's own wording (“an $N \times N$ kernel,” singular) is about programmable *values*, not a runtime-switchable *shape*.

Option B — Compile-time generic N, fully runtime-programmable coefficients

Complexity: Medium

✓ RECOMMENDED

Fix N as a VHDL generic at synthesis time (default demo value N=3), while every coefficient *value* within that N×N kernel remains freely reprogrammable at runtime — exactly what spec item 3 (“coefficients should be programmable”) literally asks for.

PROS

- Zero wasted resources — every tap in the design is a tap actually in use.
- Simplest, cleanest, fastest-timing adder tree, sized exactly right.
- A second, equally resource-optimal N=5 bitstream is just a resynthesis away — genuinely demonstrating that the RTL itself is parameterized, not hard-coded, which a judge reading the source can verify directly.

CONS

- Switching kernel *size* requires a re-synthesis rather than a live runtime toggle — acceptable, since the spec never asks for runtime size-switching.

Why recommended: Every tap you build is a tap you use, the timing story is simplest, and true VHDL generic parameterization is exactly the kind of design-quality signal the judging criteria reward — without paying Option A's real resource and risk cost for a capability the spec doesn't actually require.

DECISION

Compile-time VHDL generic for N (default demo value N=3, architecture generalized so N=5 is a resynthesis away); full runtime programmability of coefficient values within that fixed N.

11. Multi-Channel / Multi-Kernel Parallelism Architecture

HIGH FOM LEVERAGE

With every bonus mandatory, “support for multiple kernels” needs one concrete architecture. Two genuinely different readings are possible — worth weighing both before committing.

Option A — Single channel, sequentially reloadable kernel Complexity: Low

One $N \times N$ MAC array; “multiple kernels” means reloading the coefficient bank between runs and demonstrating several kernels one after another.

PROS

- Essentially free — programmable coefficients (already mandatory) already give you this.

CONS

- A weak demonstration of “multiple kernels” — really just “one kernel, reloaded,” which a strict judge may not credit as meaningfully distinct from the baseline requirement.
- Showcases no additional architectural sophistication, undercutting the goal of dominating every bonus category.

Option B — True parallel multi-channel (K shared-window MAC arrays) Complexity: High

✓ RECOMMENDED

K independent MAC/rescale/ReLU channels, each with its own kernel and its own shift value, all fed from one *shared* window generator (the line buffers and shift-register window, Section 14's genuinely expensive resource, are built once) — producing K output feature maps per cycle from a single pass over the image. This mirrors how real multi-channel CNN layers work.

PROS

- Unambiguously, substantially “multiple kernels” running in true hardware parallelism — K distinct feature maps emerge from one pass, a materially deeper demonstration than sequential reloading.
- Sharing the window generator means the expensive line-buffer resource is paid for once, not K times, keeping BRAM/LUT growth close to linear in K rather than worse.
- Natural pairing for the mandatory edge-detection demo: Sobel-X on one channel and Sobel-Y on another, computed genuinely simultaneously, enabling a gradient-magnitude edge-detection result.

CONS

- DSP/multiplier count grows linearly with K ($K \times N^2$ multiplies instead of N^2) — a real, direct FOM cost that must be weighed against Section 13's multiplier-implementation choice.
- More control-plane complexity — K independent coefficient/shift/enable register sets to address and load.

Why recommended: A genuinely deeper, more convincing demonstration of the bonus requirement, at a resource cost that stays roughly linear (not quadratic) in K because the expensive window generator is shared, not duplicated.

DECISION

Shared window-generator, K parallel independently-programmable MAC/rescale/ReLU channels. Treat K itself as a VHDL generic (consistent with Section 10's parameterization philosophy). K=2 is the minimum that credibly demonstrates true parallel multi-kernel operation (a natural Sobel-X/Sobel-Y pairing); K=4 is the recommended target if post-synthesis DSP/LUT numbers hold up — let measured FOM data pick the final value, not instinct.

12. Core Convolution Compute Topology

This is the central architectural decision. The 1-output-pixel-per-cycle requirement is mandatory and non-negotiable, so every option below is evaluated first on whether it can sustain that, and second on cost.

Option A — Direct fully-parallel broadcast-tree MAC array

Complexity: Medium

✓ RECOMMENDED

Every cycle, the current $N \times N$ window (already sitting in the Section 14 shift-register array) is multiplied element-wise against all N^2 kernel coefficients simultaneously, and a balanced binary adder tree ($\log_2(N^2)$ levels) reduces the N^2 products to one running sum per cycle.

PROS

- Conceptually the simplest topology to design *and* verify — a straight, feed-forward datapath with no feedback or complex scheduling.
- Satisfies 1 output/cycle almost by construction, since a fresh complete window arrives every cycle and the parallel tree processes it in a pipelined, feed-forward manner.
- Maps almost 1:1 onto a simple `sum(window * kernel)` NumPy expression — the easiest topology to write a bit-exact golden model for.
- For small-to-moderate N (3×3 , 5×5) the adder-tree depth is tiny (just 4 levels for 3×3), trivially pipelineable for good timing closure.

CONS

- DSP/LUT count scales as $O(N^2)$ — expensive for large N (e.g. $7 \times 7 = 49$ taps), though irrelevant at your target $N=3$.
- Not the most theoretically flops-efficient topology in the abstract.

Why recommended: For a small target N and a hard correctness + 1px/cycle bar, this topology's design and verification simplicity is worth far more than the multiply-count savings the alternatives offer at this scale.

Option B — Systolic PE-array Complexity: High

A 2D grid of tiny, identical, locally-connected processing elements, each holding one coefficient, computing one multiply-accumulate per cycle and passing a partial sum to its neighbor in a synchronized, skewed wavefront — the same fundamental pattern used in high-performance production accelerators (e.g. TPU-style arrays).

PROS

- Because every connection is to an immediate neighbor (no long wires, no big fan-in), this topology tends to place-and-route with excellent timing margins even as size grows — the critical path is always “one PE’s worth of logic,” never a whole adder tree.
- Very regular, tileable layout is friendly to Vivado’s implementation tools.
- Scales gracefully to much larger arrays than Option A can.

CONS

- Meaningfully more design and verification effort: the data-skewing/timing alignment between neighboring PEs must be exactly right — a classic systolic bug source is an off-by-one-cycle misalignment between the pixel stream and partial-sum stream.
- The golden model needs to model the same cycle-accurate dataflow to help debug an alignment issue quickly, or you verify only final outputs and lose that debug visibility.
- Its main advantage — scaling to large arrays — is a benefit you’re mostly not spending at $N=3-5$; you’d be paying extra risk and complexity to solve a scaling problem you don’t actually have.

Option C — Separable decomposition (1D row pass + 1D column pass)

Complexity: Medium–Very High

For kernels that are exactly rank-1 factorizable into a horizontal 1D filter times a vertical 1D filter (e.g. classic Sobel), running two sequential N -tap 1D convolutions instead of one N^2 -tap 2D convolution cuts multiply count from N^2 to roughly $2N$.

PROS

- Real, substantial DSP/LUT reduction when the kernel genuinely is separable — directly boosts FOM.

CONS

- **Correctness trap:** the spec requires kernel coefficients to be freely programmable with no restriction to separable (rank-1) matrices. A judge testing genuine programmability with an arbitrary, non-separable kernel would silently get a wrong answer from a separable-only engine — a failure mode that wouldn’t show up on your own demo kernels and would only appear on the judge’s stress-test data.
- Building general on-the-fly separability detection/factorization in hardware is a research-grade problem, not appropriate here; and at $N=3$ the saving (9 vs. 6 multiplies) is modest anyway relative to the added risk.

Option D — Winograd fast convolution (e.g. $F(2 \times 2, 3 \times 3)$)

Complexity: Very High

✓ RECOMMENDED

An algebraic transform technique: input tile and kernel are each passed through a fixed small-integer/fractional transform matrix (implementable as adds/shifts, not multiplies), the transformed values are multiplied element-wise (this is where the real multiply-count cut happens), and an inverse transform (again adds/shifts) produces the final output tile. For the well-known $F(2 \times 2, 3 \times 3)$ case: 4 multiplies produce 4 outputs, versus $9 \times 4 = 36$ multiplies the naive way for the same 4 outputs — roughly a 9× theoretical multiply reduction. Used in real production deep-learning accelerators for exactly this kind of multiply-count reduction.

PROS

- The single largest possible DSP-count reduction of any option here — the option most likely to produce a genuinely outstanding FOM number if executed correctly.
- A legitimately advanced, research-grade technique that would stand out enormously in a student competition report — directly in the spirit of “highest possible performance, willing to research and implement it.”

CONS

- **Numerical-accuracy risk:** the transform matrices introduce fractional intermediate values; Winograd is well-documented in the accelerator literature to have measurably larger numerical error than direct convolution at low bit-widths — exactly your regime (8-bit kernel).
- **Generality risk:** practical Winograd transforms are formulated per specific (kernel-size, tile-size) pair; a 5×5 build needs separately-derived transform matrices, not a simple generic recompile — harder to reconcile with Section 10's parameterization goal.
- **Verification difficulty:** you now need to independently verify the transform-domain intermediate arithmetic on top of the base convolution result, roughly doubling verification surface area.
- **Schedule/complexity risk:** genuinely the highest-complexity option in this document, and a real risk to the correctness/1px-cycle/bonus-completeness bar if it consumes time better spent on the lower-risk, comparably-impactful Section 13/14 FOM levers.

Why recommended: Two-tier recommendation: (1) build Option A first, fully verified, as your working, submittable, always-correct baseline — this is risk sequencing, not settling. (2) Once Option A is complete and verified, implement Winograd $F(2 \times 2, 3 \times 3)$ as a second, independent top-level architecture, verified against the exact same golden-model regression suite, and report *both* implementations' real synthesis results side by side. That empirical head-to-head comparison — measured FOM delta, measured numerical-accuracy delta, and why — is likely the single highest-leverage addition you could make to the report, precisely because most teams won't attempt it and doing it rigorously (not just claiming it) is a strong differentiator. It must never be allowed to put the core correctness/1px-cycle/bonus-completeness requirements at risk.

DECISION

Direct fully-parallel broadcast-tree MAC array (Option A) as the primary, mandatory, always-correct topology for every channel. Winograd F(2×2,3×3) (Option D) as an explicit, high-value stretch goal — a second, independently-verified architecture variant for head-to-head reporting — pursued only after the direct-form baseline is complete and fully verified. Systolic (Option B) and pure separable (Option C) are not recommended given your target N and constraints.

13. Multiplier Implementation: DSP48 vs. LUT Fabric

HIGH FOM LEVERAGE

This is one of the single highest-leverage decisions in the entire document for the stated goal of maximizing FOM, precisely because the formula's own weighting (DSPs at 50×) doesn't track a small 8-bit multiplier's real fabric-area cost particularly closely.

Option A — Infer DSP48E1 hard macros for every multiply Complexity: Low

Let Vivado map every 8×8 signed multiply onto the xc7z020's dedicated DSP48E1 slices (220 available), each natively supporting up to a 25×18 signed multiply plus a 48-bit accumulator and optional internal pipeline registers.

PROS

- Best possible timing — DSP48E1 is a hardened, pre-characterized, very fast block with built-in optional pipeline registers, making high Fmax straightforward.
- Lowest LUT usage of any multiplier implementation.
- Lowest design/verification risk — the default, most well-trodden path.
- Internal DSP accumulation can even fold part of the Section 5 adder tree into the DSP itself for free, via cascading.

CONS

- Each DSP48E1 costs 50 points in the FOM's resource-score denominator. At N=3, K=4 (36 multiplies total), that's 36 DSPs = 1,800 denominator points from DSPs alone — likely to dominate the entire resource-score term and directly suppress the reported FOM, even though the design's real silicon efficiency is excellent.

Option B — Force LUT-fabric multipliers

Complexity: Medium

✓ RECOMMENDED

Disable DSP inference via the `use_dsp = "no"` synthesis attribute, letting Vivado build each 8×8 signed multiply from ordinary LUTs and carry chains instead of the hardened DSP block.

PROS

- DSPs used drops to zero — the FOM formula's harshest-weighted term (50×) disappears entirely.
- A well-optimized 8×8 signed fabric multiplier commonly costs on the order of a few dozen LUTs — very likely under the “50 LUTs” break-even point implied by the formula's own weighting, meaning this is very likely a net FOM win over DSP-based multiplication for 8-bit operands specifically.
- Arguably the single highest-leverage individual decision in this document for your stated “maximize FOM” priority.

CONS

- Worse raw timing than DSP48E1 for the same multiply — fabric logic and carry chains are simply slower, directly threatening the 1px/cycle-at-a-good-Fmax goal unless compensated with deliberate pipelining of the multiplier itself (Section 15).
- Real risk of LUT/routing congestion once multiplied out across $N^2 \times K = 36+$ parallel multiplies, competing for fabric with the also-LUT-based line buffers and adder trees — the xc7z020's ~53,200 LUTs give real headroom, but *congestion*, not raw count, is what actually threatens timing closure, so this needs real synthesis-and-measure iteration.

Why recommended: Directly, substantially serves the stated FOM-maximization priority. See the empirical verification note below — this should be measured, not just assumed.

Option C — Hybrid (DSP for some multiplies, LUT-fabric for the rest)

Complexity: Medium

Selectively apply the `use_dsp` attribute per-instance, keeping DSPs only where their timing benefit is most needed and using fabric multiplication everywhere else.

PROS

- A tunable middle ground between Option A's timing safety and Option B's FOM aggression.

CONS

- Adds a manual, per-instance tuning dimension that needs real synthesis data to guide sensibly — premature to commit to before measuring both pure extremes.

DECISION

Recommended primary strategy: LUT-fabric multipliers (Option B). **Empirical verification note:** the `use_dsp` attribute is a one-line toggle — synthesize the Section 12 direct-form array both ways (DSP-inferred and LUT-forced), pull the real post-synthesis LUT/DSP/Fmax numbers for both, and compute the actual FOM for each. Given the formula's 50× DSP weighting, LUT-fabric is very likely to win — but the exact break-even point depends on how many LUTs your specific 8×8 multiplier synthesizes to on this exact device and Vivado version, which is a five-minute experiment, not a guess. Fall back to Option C only if pure LUT-fabric can't close timing at a competitive Fmax after genuine pipelining effort.

14. Window Generation & Line-Buffer Memory Architecture

HIGH FOM LEVERAGE

The second major FOM lever. BRAM carries the single harshest per-unit penalty (100×) of any resource in the formula — and a line buffer, at the spec's minimum 32-pixel row width, is a poor fit for a whole 18Kb block in the first place.

Option A — BRAM-based line buffers Complexity: Low

Implement each of the $N-1$ row-delay buffers as a dedicated Block RAM (18Kb/36Kb primitive), sized to hold one full image row.

PROS

- The traditional, obvious tool for “store a whole row of pixels” — simple mental model, well-understood dual-port behavior, ample capacity margin.

CONS

- Each BRAM used costs 100 FOM-denominator points — the single harshest per-unit penalty of any resource type in the entire formula.
- For a 32-pixel-wide row (or even a fairly large image), you use a tiny fraction of an 18Kb block's actual capacity while paying for the whole block — a poor efficiency trade specifically under this scoring formula.
- With Section 11's shared-window-generator design this cost isn't multiplied by K , but it's still real: $N-1 = 2$ line buffers for the shared generator alone is already 200 FOM-denominator points.

Option B — SRL-based (fabric shift-register) line buffers

Complexity: Low–Medium

✓ RECOMMENDED

Use Xilinx 7-series LUTs configured as SRL16/SRL32 shift-register primitives, built entirely from fabric with no BRAM involved — a natural fit, since a line buffer is fundamentally just a fixed-length delay line, exactly what an SRL is purpose-built for. Written as a simple VHDL shift-register process; Vivado infers SRL primitives automatically (verify this in the synthesis report rather than assuming it).

PROS

- BRAM usage for the line buffers drops to zero — eliminating the single most heavily-penalized resource category (100×) from this part of the design entirely.
- For the spec's minimum 32-pixel row (or even a generously larger demo image), this is a trivially small, extremely LUT-efficient structure — roughly one SRL32 per bit of pixel width per row, a vanishingly small cost on a device with over 50,000 LUTs.
- A pure fixed-delay access pattern is exactly what SRL provides — no wasted flexibility, no wasted cost.

CONS

- Very long shift chains (much wider images than this spec calls for) can, in principle, become a routing/timing consideration — comfortably outside the range this competition actually requires.
- Requires an explicit post-synthesis check that Vivado actually inferred SRLs rather than falling back to a flip-flop chain (which would cost far more FFs).

Why recommended: Zeroes out the harshest-weighted resource category in the entire FOM formula, at negligible design complexity, for an image size firmly within what this spec actually requires.

Option C — Distributed RAM / LUTRAM-based line buffers

Complexity: Medium

LUTs configured as small addressable memories with explicit read/write pointer management, rather than SRL's pure shift semantics.

PROS

- Like Option B, keeps BRAM usage at zero.

CONS

- A line buffer only ever needs “the pixel from exactly N columns/one row ago” — never an arbitrary address — so distributed RAM's addressing flexibility is unused overhead here.
- Typically costs somewhat more LUTs than the equivalent SRL implementation, plus needs explicit pointer-management control logic SRL doesn't need at all.

DECISION

SRL-based fabric shift-register line buffers, explicitly targeting zero BRAM usage in the window-generation datapath. Combined with Section 13's LUT-fabric multiplier recommendation, this design direction targets a PL-side BRAM count of essentially zero and a DSP count of essentially zero — attacking both of the FOM formula's harshest-weighted terms simultaneously. Caveat: if the Section 18 AXI-DMA data path is adopted, verify via the post-synthesis utilization report whether the DMA IP itself instantiates any small internal BRAM/FIFO in the PL fabric — a separate, largely unavoidable cost outside the accelerator core itself.

15. Pipelining & Retiming Strategy

Note up front: FOM's "throughput" term is pixels/*cycle*, not pixels/second — so pipelining depth doesn't change the FOM numerator directly. It matters for a different, equally real reason: the competition's overall judging criteria explicitly include "latency, throughput, timing closure" alongside FOM, and a design that can't close timing at a competitive frequency is genuinely weaker on those criteria even with a maximal FOM number.

Option A — Fully combinational per-channel datapath Complexity: Low

Let the whole per-cycle computation (multiply → adder tree → rescale → saturate → ReLU) happen as one long combinational path, registering only at the very output.

PROS

- Simplest possible control logic — no pipeline-stage bookkeeping.

CONS

- A long combinational path (especially with Section 13's LUT-fabric multipliers, which are inherently slower than DSP48E1) directly threatens Fmax and, in the worst case, threatens whether timing closes at all at a usable frequency.
- Compounds badly with the LUT-fabric multiplier decision — skipping pipelining there stacks two timing risks on top of each other.

Option B — Deep, maximal pipelining Complexity: High

Insert a pipeline register at essentially every opportunity — after each partial product, every adder-tree level, rescale, saturate, and ReLU — prioritizing maximum Fmax above all else.

PROS

- Best possible Fmax and timing margin.

CONS

- FF cost is real but irrelevant to FOM (FFs aren't in the formula). The real cost is design/verification: every added stage adds latency that must stay correctly synchronized with any control signal traveling alongside the data (e.g. Section 11's per-channel enable bits), an easy source of subtle bugs.
- Past a certain point, additional stages give diminishing Fmax returns while continuing to add synchronization complexity — over-pipelining a datapath with an already-short natural critical path (Section 12's N=3 direct-form array has only ~4 adder-tree levels) spends complexity budget for a return already mostly captured.

Option C — Targeted, structurally-placed pipelining + tool-assisted retiming

Complexity: Medium

✓ RECOMMENDED

Place 3–5 pipeline registers at the datapath's natural structural seams (after the multiply stage, after 1–2 adder-tree levels, after rescale+satuate, after ReLU — conveniently, these seams are also the sub-block boundaries you're already building and verifying independently), then let Vivado's register-retiming optimization fine-tune exact register placement within each stage.

PROS

- Captures most of the achievable Fmax benefit without Option B's full bookkeeping burden.
- Keeps latency (cycles input-to-output) predictable and modest, which matters directly for the Table 1 latency figure.
- Explicit use of retiming is a legitimate, easy “digital design quality” talking point for the report — deliberate tool-assisted optimization, not brute-force register-stuffing.

CONS

- Still requires at least one real synthesis iteration — pipeline at the seams first, look at the actual critical path in the timing report, and only add more stages where a specific path is still failing, rather than guessing the right count up front.

Why recommended: Captures most of the realistic Fmax benefit at a fraction of Option B's bookkeeping cost, and turns pipeline placement into a measured decision rather than a guessed one.

DECISION

Structural/balanced pipelining (3–5 stages at natural sub-block boundaries) as the starting point, refined by real post-synthesis timing data and Vivado's register-retiming optimization — a measurement-driven process, not a fixed prescription.

16. ReLU Implementation & Control

The spec calls ReLU an optional post-processing step. The decision here is simply whether it's always-on or independently controllable.

Option A — Hard-wired always-on ReLU Complexity: Low

Every output unconditionally passes through $\max(0, x)$ — the sign bit (MSB) of the final signed value gates a single mux.

PROS

- Simplest possible implementation — a single mux and inverter, essentially free.

CONS

- The spec frames ReLU as operator-optional, not a given — an always-on design can never show the raw pre-ReLU convolution output, which for some edge-detection kernels carries meaningful negative-value information you might want to display in the report.

Option B — Runtime-toggleable ReLU Complexity: Low ✓ RECOMMENDED

Identical mux to Option A, but its select input comes from a 1-bit enable register loaded alongside each channel's kernel/shift configuration, instead of being hardwired.

PROS

- Costs one more control-register bit over Option A — a single flip-flop, negligible.
- Full flexibility to show both raw and ReLU'd hardware output from the same design, directly useful for the report's design-tradeoff discussion.
- Different parallel channels (Section 11) can reasonably want different ReLU settings — e.g. keep raw signed gradient direction on one Sobel channel, ReLU a magnitude/detection channel.

CONS

- None of substance.

Why recommended: More flexibility for essentially zero cost — not a real trade-off.

DECISION

Per-channel runtime-toggleable ReLU, controlled by a 1-bit enable register loaded alongside each channel's kernel coefficients and rescale shift.

17. Control-Plane Interface

How the Zynq PS (ARM side) loads kernel coefficients, shift values, and enable bits into the PL accelerator — a low-bandwidth, infrequent-access role, distinct from the high-bandwidth pixel data path (Section 18).

Option A — AXI4-Lite register-mapped control

Complexity: Low–Medium

✓ RECOMMENDED

A standard, memory-mapped register interface (kernel-coefficient loads, per-channel shift values, per-channel ReLU-enable bits, start/stop/status flags) accessed over the industry-standard AXI4-Lite protocol — the natural, tool-supported way for the Zynq PS to talk to custom PL logic, directly wireable in Vivado's IP Integrator.

PROS

- The standard, expected pattern for exactly this PS–PL control scenario on a Zynq device.
- Vivado auto-generates the address-map documentation needed for the report.
- Trivially readable/writable from Python via simple memory-mapped reads/writes — matches your stated toolchain directly.
- AXI4-Lite's modest bandwidth is a complete non-issue here since kernel loads happen once per run, not every cycle.

CONS

- Standard IP Integrator setup work — not a novel design problem, but real integration effort.

Why recommended: The idiomatic, tool-supported, directly Python-friendly answer for exactly this role on this platform.

Option B — Raw GPIO/parallel-register control

Complexity: Low

Wire PS GPIO pins directly to control registers, with no AXI protocol involved.

PROS

- Marginally simpler to bring up for a very first smoke test.

CONS

- Doesn't scale to the amount of individually-addressable state actually needed (K channels $\times$ coefficients + shift + ReLU-enable each).
- Not the idiomatic pattern for a Zynq SoC design — a judge evaluating digital design quality on a Zynq platform would reasonably expect AXI, not bit-banged GPIO.
- Doesn't integrate as cleanly with a Python-driven host application.

DECISION

AXI4-Lite register-mapped control interface, integrated via Vivado IP Integrator against the Zynq Processing System.

18. Data-Plane Interface for the ZedBoard Demo

How pixel data actually moves at speed during the board demo. This decision directly determines whether your hard-won 1-pixel/cycle core is actually measurable as such on real hardware, or invisibly bottlenecked by something else.

Option A — AXI4-Lite register-polled pixel feeding Complexity: Low

Reuse the Section 17 control interface for data movement too — write one pixel via a register write, read one output pixel via a register read, repeat.

PROS

- Zero additional IP beyond what Section 17 already needs — simplest possible bring-up path.

CONS

- Catastrophically defeats the entire point of a 1-pixel-per-cycle accelerator: AXI4-Lite register access from PS software carries per-transaction software/bus overhead many cycles wide, so measured hardware throughput would be bottlenecked by the polling loop, not by your actual accelerator core.
- Would make your real, hard-won 1px/cycle achievement invisible in any on-board-measured throughput number — actively undermining your own strongest technical result.

Option B — AXI4-Stream + AXI DMA (PS DDR ↔ PL accelerator)

Complexity: Medium-High

✓ RECOMMENDED

Store test images in PS-side DDR memory (loaded there by your Python control application), then use a DMA engine (Vivado's `axi_dma/axi_datamover` IP) to autonomously stream pixels into the accelerator's AXI4-Stream input port at full accelerator speed with no per-pixel software involvement, streaming results back to a DDR output buffer the same way.

PROS

- The only data-plane option that lets the hardware actually run at its true 1px/cycle rate during the demo *and be measured doing so*.
- The technically correct, idiomatic Zynq dataflow pattern for a sustained-throughput, PL-compute-bound task — matches how real production CNN accelerators on this class of device move data.
- On-board-measured throughput reflects real accelerator performance, not a software artifact.

CONS

- Meaningfully more integration work: correctly configuring and driving the DMA engine (descriptor setup, interrupt or polled completion handling) from PS software, and correctly implementing the AXI4-Stream handshake (`tvalid/tready/tlast`) on the accelerator's boundary — a real but standard, well-documented interface.
- As flagged in Section 14, worth confirming post-synthesis whether the DMA IP itself consumes a small amount of PL-side BRAM for internal buffering.

Why recommended: Effectively non-negotiable given that 1 output pixel/cycle is explicitly your must-have requirement: that requirement is only meaningful, demonstrable, and measurable if the surrounding data plane doesn't itself become the bottleneck. Building an excellent core and then feeding it through Option A would waste the core's real achievement.

Option C — Xilinx Video DMA (VDMA) / video-formatted AXI4-Stream

Complexity: High

Use Xilinx's video-specific DMA IP and video-timing-aware AXI4-Stream conventions (frame sync, line sync, active-video windows), designed for camera/display pipelines.

PROS

- Reuses well-tested Xilinx video IP, if video-format compatibility were independently needed for some other reason.

CONS

- Pulls in a whole video-timing convention your task (a single static grayscale image/feature map, not a live video stream) doesn't need at all.
- Meaningfully more IP integration overhead and conceptual complexity than plain AXI DMA, for zero benefit specific to this competition's requirements.

DECISION

AXI4-Stream input/output ports on the accelerator core, fed via AXI DMA to/from PS DDR memory, with AXI4-Lite (Section 17) reserved strictly for control/configuration.

19. PS-Side Software Enablement for Python

Since the ZedBoard is a Zynq PS+PL SoC, getting Python to actually drive the accelerator (register control, DMA orchestration, golden-model comparison) requires deciding what runs on the ARM PS cores.

Option A — PetaLinux (custom embedded Linux on the PS)

Complexity: High

✓ RECOMMENDED

Build a custom bootable embedded-Linux BSP (kernel, root filesystem, device tree) tailored to your specific Vivado hardware design using Xilinx's PetaLinux toolchain; once running, it's a real Linux environment on the ZedBoard's ARM cores, meaning a genuine Python interpreter runs directly on the board, with UIO//dev/mem-based access to your AXI4-Lite registers and standard Linux DMA-buffer APIs for the AXI DMA path.

PROS

- Python runs natively, physically on the ZedBoard — no separate host PC needed for the actual demo once built.
- Cleanly matches your stated "Python for the software side" preference in the most literal, complete sense.
- A real, familiar Linux environment (file system, standard libraries, easy to add NumPy for on-target golden-model comparison) for the most self-contained, impressive demo.

CONS

- Genuinely substantial one-time toolchain investment: hardware-platform export from Vivado, kernel config, device-tree edits for your custom AXI peripherals, root-filesystem build — necessary plumbing, not accelerator-design work.

Why recommended: Gets Python running natively on the board, most completely matching your stated software-side language preference, with a well-trodden (if effort-intensive) official build path — unlike Option C below.

Option B — Bare-metal C application + host-PC Python

Complexity: Medium

Skip embedded Linux; write a small standalone C application (via Vitis's bare-metal BSP, auto-generated from your Vivado hardware platform) that does the AXI-Lite/DMA plumbing on-target, driven/orchestrated by a Python script on your laptop over UART or a similar link.

PROS

- Much faster to bring up than a full PetaLinux build — no kernel/rootfs build cycle, no Linux driver model to navigate — a real schedule/risk reduction if time is tighter than it looks.

CONS

- The part that actually talks to the hardware ends up being C, not Python — only partially matching your stated preference, with Python relegated to a thin host-side orchestration role over a serial link.
- Requires a host PC as part of the demo setup, rather than a fully self-contained board.

Option C — PYNQ

Complexity: Very High

Xilinx's Python-productivity framework for Zynq — Jupyter notebooks and a Python overlay-loading API, giving the most Python-native development experience possible, normally used on officially-supported boards.

PROS

- If it works, the single nicest day-to-day Python development experience of any option here.

CONS

- Verified directly: the ZedBoard is explicitly not one of PYNQ's officially supported boards (only specific boards like the PYNQ-Z1/Z2 ship pre-built images). Using PYNQ on a ZedBoard requires manually building a custom image — extracting/upgrading an old Avnet ZedBoard board-support package, running it through PetaLinux, then building the full PYNQ Linux/Jupyter stack on top.
- This is genuinely *more* total build effort than Option A's plain PetaLinux path, while landing in essentially the same place (Python via Linux on the PS) plus an unneeded Jupyter layer.
- Community ZedBoard PYNQ build guides are commonly tied to older, specific Vivado/PetaLinux versions, which may not cleanly match Vivado 2025.2 — real risk of spending research time on Linux-build-system archaeology instead of the accelerator.

DECISION

PetaLinux-based embedded Linux on the PS, running a native Python interpreter on-target for the full demo application (register control, DMA orchestration, golden-model comparison). Option B (bare-metal + host Python) kept as a lower-risk fallback if the PetaLinux build timeline becomes a genuine schedule threat.

20. Master Recommendation Table

Every decision in this document, in one place, for quick reference. This is a starting point for your own decisions, not a substitute for reading the reasoning above — several of these (Sections 12 and 13 especially) are explicitly flagged as needing empirical, measured confirmation rather than being final on paper alone.

Decision	Recommended Choice	Ref
Input pixel format	8-bit unsigned integer (Q8.0)	§3
Kernel format	Raw signed 8-bit integer; format set by rescale shift	§4
Accumulator width	Worst-case formula: $\text{pixel_w} + \text{kernel_w} + \text{ceil}(\log_2(N^2)) + 1$	§5
Rescale shift	Programmable per-channel	§6
Rounding	Round-half-up	§7
Overflow handling	Saturate	§8
Boundary convention	“Valid” convolution (no padding)	§9
Kernel-size (N)	Compile-time VHDL generic, default N=3	§10
Multi-kernel parallelism	K shared-window parallel channels, target K=4	§11
Compute topology	Direct parallel MAC array (baseline) + Winograd F(2,3) stretch goal	§12
Multiplier implementation	LUT-fabric (use_dsp=no), empirically verified vs. DSP48	§13
Line-buffer memory	SRL-based fabric shift registers (zero BRAM target)	§14
Pipelining	3–5 structural stages + Vivado retiming	§15
ReLU	Per-channel runtime-toggable	§16
Control interface	AXI4-Lite register map	§17
Data interface	AXI4-Stream + AXI DMA (PS DDR ↔ PL)	§18
PS software stack	PetaLinux + native on-target Python	§19

21. Projected Table 1 Preview

A preview of the competition's required results table, populated with the recommended specification values from this document. Every “TBD” entry is a number that can only come from real synthesis/simulation — not something to fill in on paper.

Parameter	Recommended / Target Value	Comments
Input image size	32×32 minimum (generic W×H)	“Valid” conv → output (W−2)×(H−2) for N=3
Input precision	8-bit unsigned integer (Q8.0)	Native grayscale representation
Kernel precision	8-bit signed integer	Format carried by per-channel programmable shift
Architecture type	Direct parallel MAC array, K=4 shared-window channels	Winograd variant reported separately if completed
Multipliers / MACs	$N^2 \times K = 9 \times 4 = 36$ (N=3, K=4)	Target: 0 DSPs (LUT-fabric), pending empirical measurement
Pipeline stages	3–5 (structural, retiming-assisted)	Final count set by post-synthesis timing data
Latency	Pipeline depth + window-fill cycles	To be measured in simulation
Throughput	1 output pixel/cycle per channel (4 total)	Mandatory, non-negotiable
FPGA utilization	Target: 0 BRAM, 0 DSP	To be confirmed by synthesis
Maximum frequency	TBD	Depends on §15 pipelining outcome
Power estimate	TBD	Vivado power report, post-implementation
Verification status	Golden-model bit-exact match (planned)	Python/NumPy fixed-point model
FOM	TBD — target: maximize via 0-BRAM/0-DSP strategy	Computed post-synthesis

Prepared as a decision-support document. Items marked TBD require real synthesis/simulation data and cannot be responsibly filled in on paper alone.